ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Design Patterns

Week 1: Introduction and Singleton

Vang Le

(lvvang@it.tdt.edu.vn)

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - **What is a Design Pattern**
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

What is a Design Pattern?

- A problem that someone has already solved.
- A model or design to use as a guide
- More formally: “A proven solution to a common problem in a specified context.”

Real World Examples

- Blueprint for a house
- Manufacturing

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - **Why Study Design Patterns**
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

Why Study Design Patterns?

- Provides software developers a toolkit for handling problems that have already been solved.
- Provides a vocabulary that can be used amongst software developers.
  - The Pattern Name itself helps establish a vocabulary
- Helps you *think* about how to solve a software problem.

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - **History of Design Patterns**
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

History of Design Patterns

- Christopher Alexander (Civil Engineer) in 1977 wrote
  - "Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, *without ever doing it the same way twice*."

- Each pattern has the same elements
  - ***Pattern Name*** – helps develop a catalog of common problems
  - ***Problem*** – describes when to apply the pattern. Describes problem and its context.
  - ***Solution*** – Elements that make up the design, their relationships, responsibilities, and collaborations.
  - ***Consequences*** – Results and trade-offs of applying the pattern

History (continued)

- In 1995, the principles that Alexander established were applied to software design and architecture. The result was the book:

"Design Patterns: Elements of Reusable Object-Oriented Software" by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.

Also commonly known as "The Gang of Four".

The Gang of Four

- Defines a Catalog of different design patterns.
- Three different types
  - *Creational* – "creating objects in a manner suitable for the situation"
  - *Structural* – "ease the design by identifying a simple way to realize relationships between entities"
  - *Behavorial* – "common communication patterns between objects"

The Gang of Four: Pattern Catalog

Creational

- **Abstract Factory**
- **Builder**
- **Factory Method**
- **Prototype**
- **Singleton**

Structural

- **Adapter**
- **Bridge**
- **Composite**
- **Decorator**
- **Façade**
- **Flyweight**
- **Proxy**

Behavioral

- **Chain of Responsibility**
- **Command**
- **Interpreter**
- **Iterator**
- **Mediator**
- **Memento**
- **Observer**
- **State**
- **Strategy**
- **Template Method**
- **Visitor**

Patterns in red will be discussed in class.

Common Pitfall

"I just learned about Design Pattern XYZ. Let's use it!"

Reality: If you are going to use a Design Pattern, you should have a reason to do so.

The *software requirements* should really drive why you are going to use (or not use) a Design Pattern.

The Book

- "Head First Design Patterns"
  - Eric Freeman & Elisabeth Freeman
- Book is based on the Gang of Four design patterns.
- Easier to read.
- Examples are fun, but not necessarily "real world".

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - **Logger Example**
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

Example: Logger

What is wrong with this code?

```
public class Logger
{
    public Logger() { ... }

    public void LogMessage(string msg){
        //Open File "log.txt"
        //Write Message
        //Close File
    }
}
```

Example: Logger

in class A:

....

Logger log = new Loger();

log.LogMessage('message1');

....

in class B:

....

Logger log = new Loger();

log.LogMessage('message2');

....

in class C:

....

Logger log = new Loger();

log.LogMessage('message3');

....

in class D:

....

Logger log = new Loger();

log.LogMessage('message4');

....

Example: Logger (cont)

- Since there is an external Shared Resource ("log.txt"), we want to closely control how we communicate with it.

- We shouldn't have to create the Logger class every time we want to access this Shared Resource. Is there any reason to?

- We need ONE.

Singleton

- GoF Definition: “The Singleton Pattern ensures a class has only *one instance*, and provides a global point of access to it.”

- Best Uses
  - Logging
  - Caches
  - Registry Settings
  - Access External Resources
    - Printer
    - Device Driver
    - Database

Logger – as a Singleton

See pg 173 in book

```
public class Logger
{
    private Logger{}

    private static Logger uniqueInstance;

    public static Logger getInstance()
    {

          if (uniqueInstance == null)
             uniqueInstance = new Logger();

          return uniqueInstance;

    }
}
```

Note the parameterless constructor

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - **Chocolate Factory Example**
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

The Chocolate Factory

- Everyone knows that all modern chocolate factories have computer controlled chocolate boilers. The job of the boiler is to take in chocolate and milk, bring them to a boil, and then pass them on to the next phase of making chocolate bars.
- Here's the controller class for Choc-O-Holic, Inc.'s industrial strength Chocolate Boiler. You'll notice they've tried to be very careful to ensure that bad things don't happen, like filling the boiler when it's already full, or boiling an empty boiler!

The Chocolate Factory

```
public class ChocolateBoiler {
        private boolean empty;
        private boolean boiled;
        public ChocolateBoiler() {
                empty = true;
                boiled = false;
        }
        public boolean isEmpty() {
                return empty;
        }
        public boolean isBoiled() {
                return boiled;
        }
}
```

The Chocolate Factory

```
public void fill() {
    if (isEmpty()) {
        empty = false;
        boiled = false;
        //control filling
    }
}
```

```
public void drain() {
    if (!isEmpty() && isBoiled()) {
        //drain the boiled milk and chocolate
        empty = true;
    }
}
public void boil() {
    if (!isEmpty() && !isBoiled()) {
        // bring the contents to a boil
        boiled = true;
    }
}
```

The Chocolate Factory

- Choc-O-Holic has done a decent job of ensuring bad things don't happen, don't ya think?
- How might things go wrong if more than one instance of ChocolateBoiler is created in an application?

```
public class ChocolateBoiler {
    private boolean empty;
    private boolean boiled;

    ChocolateBoiler() {
        empty = true;
        boiled = false;
    }

    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
            // fill the boiler with a milk/chocolate mixture
        }
    }
    // rest of ChocolateBoiler code...
}
```

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - **Lazy Instantiation**
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

Lazy Instantiation

- Objects are only created when it is needed
- Helps control that we’ve created the Singleton just once.
- If it is resource intensive to set up, we want to do it once.

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - **Singleton vs. Static Variables**
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

Singleton vs. Static Variables

- What if we had ***not*** created a Singleton for the Logger class??

- Let's pretend the Logger() constructor did a lot of setup.

In our main program file, we had this code:

```
public static Logger MyGlobalLogger = new Logger();
```

- All of the Logger setup will occur regardless if we ever need to log or not.

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - **Threading: Simple, Double-Checked, Eager Initialization**
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

Threading

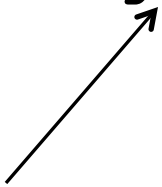
```
public class Singleton
{
   private Singleton() {}

   private static Singleton uniqueInstance;
   public static Singleton getInstance()
   {

     if (uniqueInstance == null)
           uniqueInstance = new Singleton();

       return uniqueInstance;

   }
}
```



What would happen if two different threads accessed this line at the same time?

```
public static ChocolateBoiler
        getInstance() {

    if (uniqueInstance == null) {

        uniqueInstance =
            new ChocolateBoiler();

    }

    return uniqueInstance;

}
```

Thread One	Thread Two	Value of uniqueInstance

BE the JVM

Thead One	Thead Two	Value of uniqueInstance
`public static ChocolateBoiler getInstance() {`		null
	`public static ChocolateBoiler getInstance() {`	null
`if (uniqueInstance == null) {`		
	`if (uniqueInstance == null) {`	null
`uniqueInstance = new ChocolateBoiler();`		<object1>
`return uniqueInstance;`		<object1>
	`uniqueInstance = new ChocolateBoiler();`	<object2>
	`return uniqueInstance;`	<object2>

Uh oh, this doesn't look good!

Two different objects are returned! We have two ChocolateBoiler instances!!!

Option #1: Simple Locking

```
public class Singleton
{
   private Singleton() {}

   private static Singleton uniqueInstance;

   public static synchronized Singleton getInstance()
   {
         if (uniqueInstance == null)
               uniqueInstance = new Singleton();
         return uniqueInstance;
   }
}
```

Option #2 – Double-Checked Locking

pg 182

```
public class Singleton
{
   private Singleton() {}

   private volatile static Singleton uniqueInstance;

   public static Singleton getInstance()
   {
        if (uniqueInstance == null) {
            synchronized(Singleton.class) {
               if (uniqueInstance == null)
                    uniqueInstance = new Singleton();
            }
        }

        return uniqueInstance;
   }
}
```

Option #3: “Eager” Initialization

```
public class Singleton
{
   private Singleton() {}

   private static Singleton uniqueInstance = new
  Singleton()

   public static Singleton getInstance()
   {
      return uniqueInstance;
   }
}
```

pg 181

Runtime guarantees that this is thread-safe

1. Instance is created the first time any member of the class is referenced.
2. Good to use if the application always creates; and if little overhead to create.

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - **Lab**
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

Lab #1: Turn a class into a Singleton

```
public class Logger {

    public Logger() { }

    public void WriteLine(string text) { ... }

    public string ReadEntireLog()
    {
        return "Log Entries Here";
    }
}
```

Take this class and turn it into a Singleton.

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Lab #1 Answer

Lab #2

```
public class BaseSingleton {
    private BaseSingleton() { ... }
    private static BaseSingleton instance;
    public static BaseSingleton getInstance()
    {
        if (instance == null) {
                instance = new BaseSingleton();
        }
        return instance;
    }
    //Some state variables
    protected int someInt;

    //Function is marked as virtual so that it can be overidden
    public void DoSomething() {
        someInt = 1;
    }
}
```

Lab #2 (cont)

```
public class SubClassSingleton
                    extends BaseSingleton
{
    private SubClassSingleton() { ... }

    public void DoSomething() {
        someInt = 2;
    }

    public void NewFunction() {
        //new functionality here
    }
}
```

Lab #2 (cont)

- *Question #1:* What is wrong with the constructor for SubClassSingleton?

Lab #2 (cont)

Here is the code that calls the Singleton:

```
public class Main
{
    public static void DoStuff()
    {
01      BaseSingleton.getInstance().DoSomething();

02      SubClassSingleton.getInstance().DoSomething();

03      SubClassSingleton.getInstance().NewFunction();
    }

}
```

Lab #2 (cont)

Question #2: For Line 01, what is the value of someInt after it is called?

Question #3: For Line 02, what is the value of someInt after it is called?

Question #4: What is wrong with Line 03?

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Lab #2 Answers

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - **Inheritance**
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - Summary

Inheritance Summary

- The sub-class can share state with the base class. Now you have two objects that share state.
- "Gang of Four" recommends usages of a registry of Singletons. The base class reads an environment variable to determine which Singleton to use.
- Bottom Line: Singletons and Inheritance do not mix well.

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - **Add Registry to Singleton**
  - Dependencies
  - Unit Testing
  - Summary

Add Registry to Singleton

```
public class BaseSingleton
{
   protected BaseSingleton() { }
   private static HashMap map = new HashMap();
   public static BaseSingleton getInstance(String classname)
   {
      //First, attempt to get Singleton from HashMap
      BaseSingleton singleton =
                          (BaseSingleton)map.get(classname);
      if (singleton != null)
           return singleton;
      else
      {
         //Singleton not found
         if (classname.Equals("SubClassSingleton"))
              singleton = new SubClassSingleton();
         else {......}

         //Add singleton to HashMap so we can get it again
         map.put(classname, singleton);

         return singleton;
      }
```

SingletonRegistry Class

Source

- http://www.javaworld.com/javaworld/jw-04-2003/jw-0425-designpatterns.html?page=6

Describes a SingletonRegistry whose purpose is to store Singletons!

```
public static Singleton getInstance() { return
   (Singleton)SingletonRegistry.REGISTRY.getInstance(classname); }
```

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - **Dependencies**
  - Unit Testing
  - Summary

Case Study: Dependencies

```
public class Searcher
{
  //Singleton Setup code Here

  public Collection FindCustomers(String criteria) {
     String conStr =
            GetConnectionString();
     OracleConnection conn =
              new OracleConnection(conStr);
     //Query database using criteria
     return (Collection of customers);
  }
}
```

Case Study: Dependencies

To search the database, the client code would need to do the following:

```
Searcher.getInstance().FindCustomers("Johny")
```

What happens if we need to change the database to SQL Server? Or change how we get the connection string?

Case Study: Dependencies

- The Singleton is *tightly coupled* to Oracle. If the database changed in the future, this object would need to be changed. It is also tightly coupled in how it retrieves the connection string.

- The Singleton hides object dependencies (Oracle and registry). Anyone using the Singleton would need to inspect the internals to find out what is really going on.

- Possibly memory leak since the Singleton may hold onto the resource for an infinite amount of time.

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - **Unit Testing**
  - Summary

Unit Testing

There are many problems with unit testing a Singleton.

Problem: If you are running multiple unit tests that accesses the same Singleton, the Singleton will retain state between unit tests. This may lead to undesired results!

Solution : Use Reflection to get access to the private static instance variable. Then set it to null. This should be done at the end of each unit test.

```
private static Singleton uniqueInstance;
```

Scott Densmore "Why Singletons are Evil"

- "…the dependencies in your design are hidden inside the code, and not visible by examining the interfaces of your classes and methods. You have to inspect the code to understand exactly what other objects your class uses.
- "Singletons allow you to limit creation of your objects... you are mixing two different responsibilities into the same class."
- "Singletons promote tight coupling between classes."
- "Singletons carry state with them that last as long as the program lasts…Persistent state is the enemy of unit testing."

- Source: http://blogs.msdn.com/scottdensmore/archive/2004/05/25/140827.aspx

Jeremy D. Miller "Chill out on the Singleton…"

"Using a stateful singleton opens yourself up to all kinds of threading issues. When you screw up threading safety you can create really wacky[lạ] bugs that are devilishly hard to reproduce.

My best advice is to not use caching via static members until you absolutely have to for performance or resource limitation reasons."

- Source: http://codebetter.com/blogs/jeremy.miller/archive/2005/08/04/130302.aspx

Comments from Others

- “I had cases where I wanted to create a subclass, and the singleton kept referring to the superclass.”
- “..writing a good unit test was impossible because the class invoked a singleton, and there was no way to inject a fake.”
- “Unit testing a singleton is not so much a problem as unit testing the other classes that use a singleton. They are bound so tightly that there is often no way to inject a fake (or mock) version of the singleton class. In some cases you can live with that, but in other cases, such as when the singleton accesses an external resource like a database, it's intolerable for a unit test.”

Other Useful Articles

- Singleton Explained
  - http://c2.com/cgi/wiki?SingletonPattern
  - http://www.oodesign.com/singleton-pattern.html

- Unit Testing
  - http://imistaken.blogspot.com/2009/11/unit-testing-singletons.html
  - http://weblogs.asp.net/okloeten/archive/2004/08/19/217182.aspx

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

Agenda

- Introduction to Design Patterns
  - What is a Design Pattern
  - Why Study Design Patterns
  - History of Design Patterns
- The Singleton Pattern
  - Logger Example
  - Chocolate Factory Example
  - Lazy Instantiation
  - Singleton vs. Static Variables
  - Threading: Simple, Double-Checked, Eager Initialization
  - Lab
  - Inheritance
  - Add Registry to Singleton
  - Dependencies
  - Unit Testing
  - **Summary**

ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

SUMMARY

- ***Pattern Name*** – Singleton
- ***Problem*** – Ensures one instance of an object and global access to it.
- ***Solution***
  - Hide the constructor
  - Use static method to return one instance of the object
- ***Consequences***
  - Lazy Instantiation
  - Threading
  - Inheritance issues
  - Hides dependencies
  - Difficult unit testing